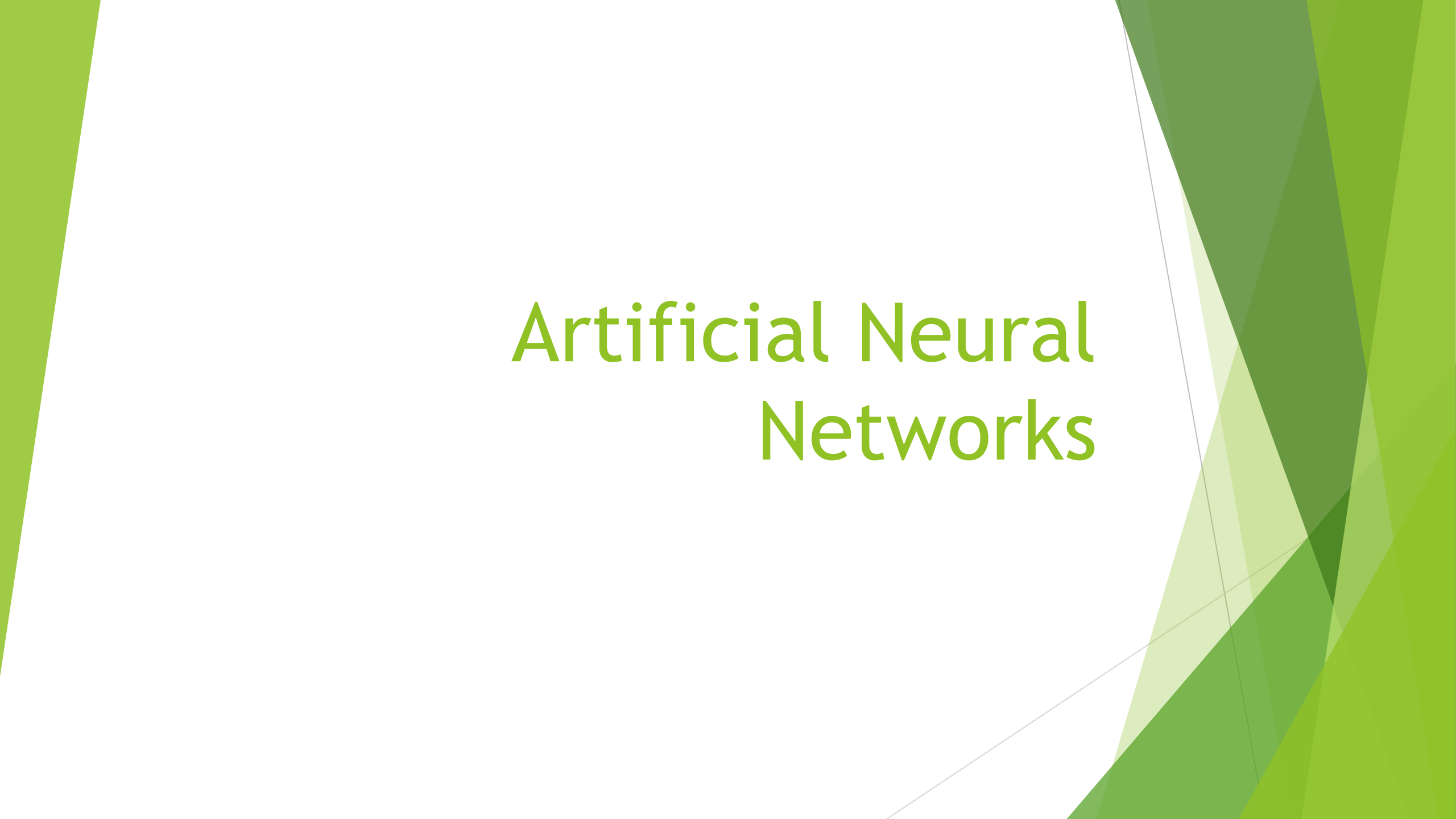


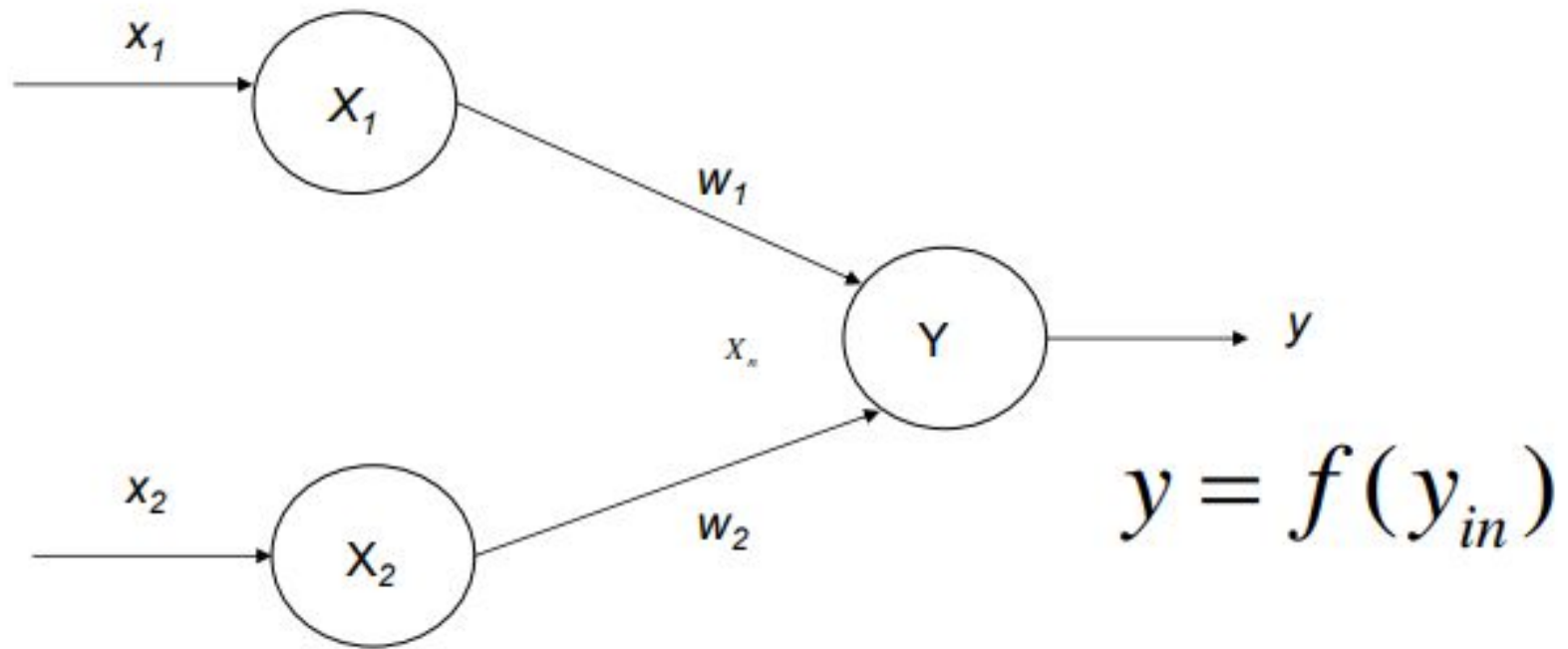
The background features abstract, overlapping green geometric shapes, primarily triangles and polygons, in various shades of green, creating a modern and dynamic visual effect.

Artificial Neural Networks

Neural Networks

- ▶ NNs are constructed and implemented to model the human brain.
- ▶ Performs various tasks such as pattern matching, classification, optimization function, approximation, vector quantization and data clustering.
- ▶ These tasks are difficult for traditional computers.
- ▶ ANN possess a large number of processing elements called nodes/neurons which operate in parallel.
- ▶ Neurons are connected with others by connection link.
- ▶ Each link is associated with weights which contain information about the input signal.
- ▶ Each neuron has an internal state of its own which is a function of the inputs that neuron receives- Activation level.

Neural Networks



$$y_{in} = x_1 w_1 + x_2 w_2$$

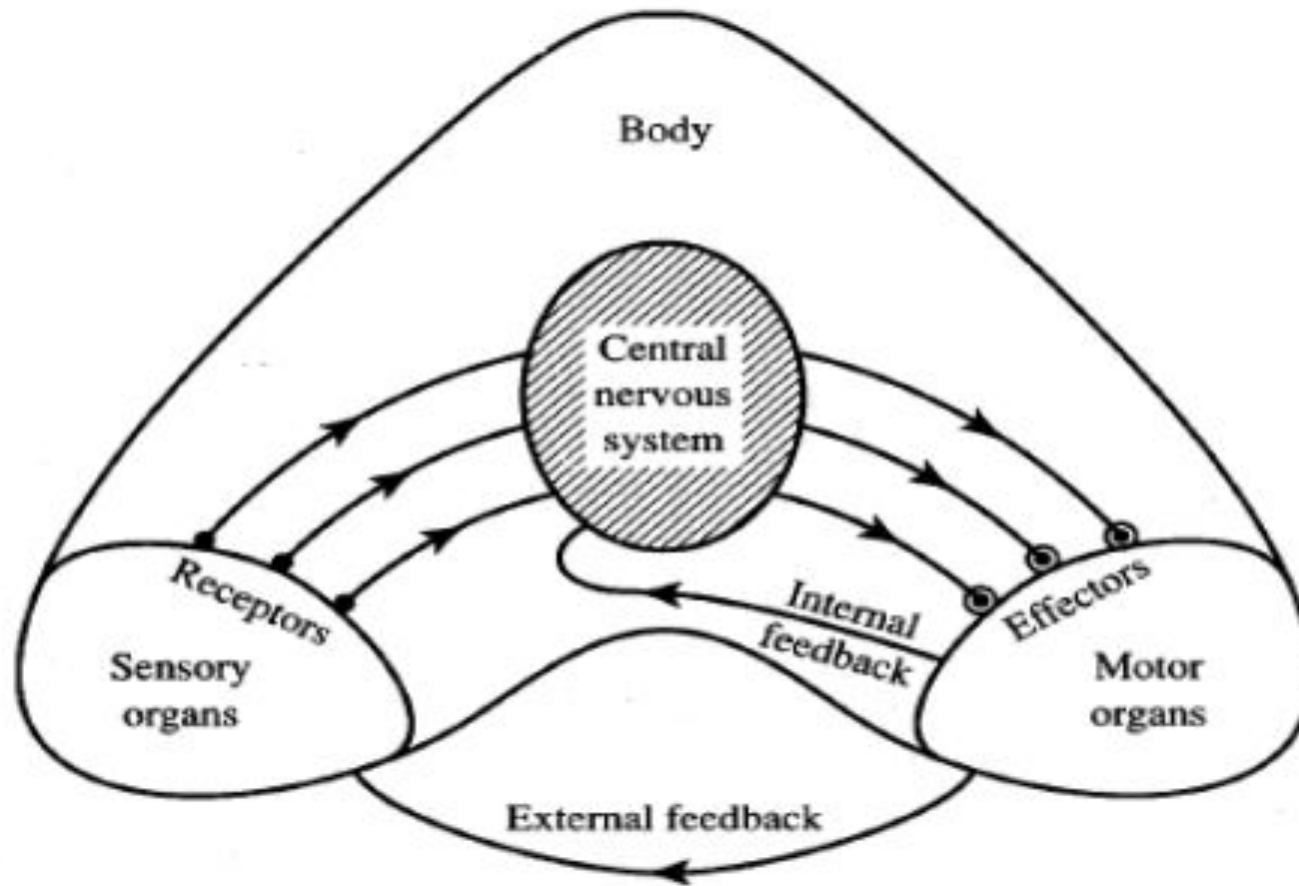
NN for linear equation

- A linear equation is an algebraic equation where each term is a constant or the product of a constant and a single variable raised to the first power, forming a straight line when graphed. Its standard forms include $(Ax + B = 0)$ (one variable) or $(Ax + By = C)$ (two variables), representing a relationship where variables change at a constant rate, like $(y = mx + b)$ (slope-intercept form).

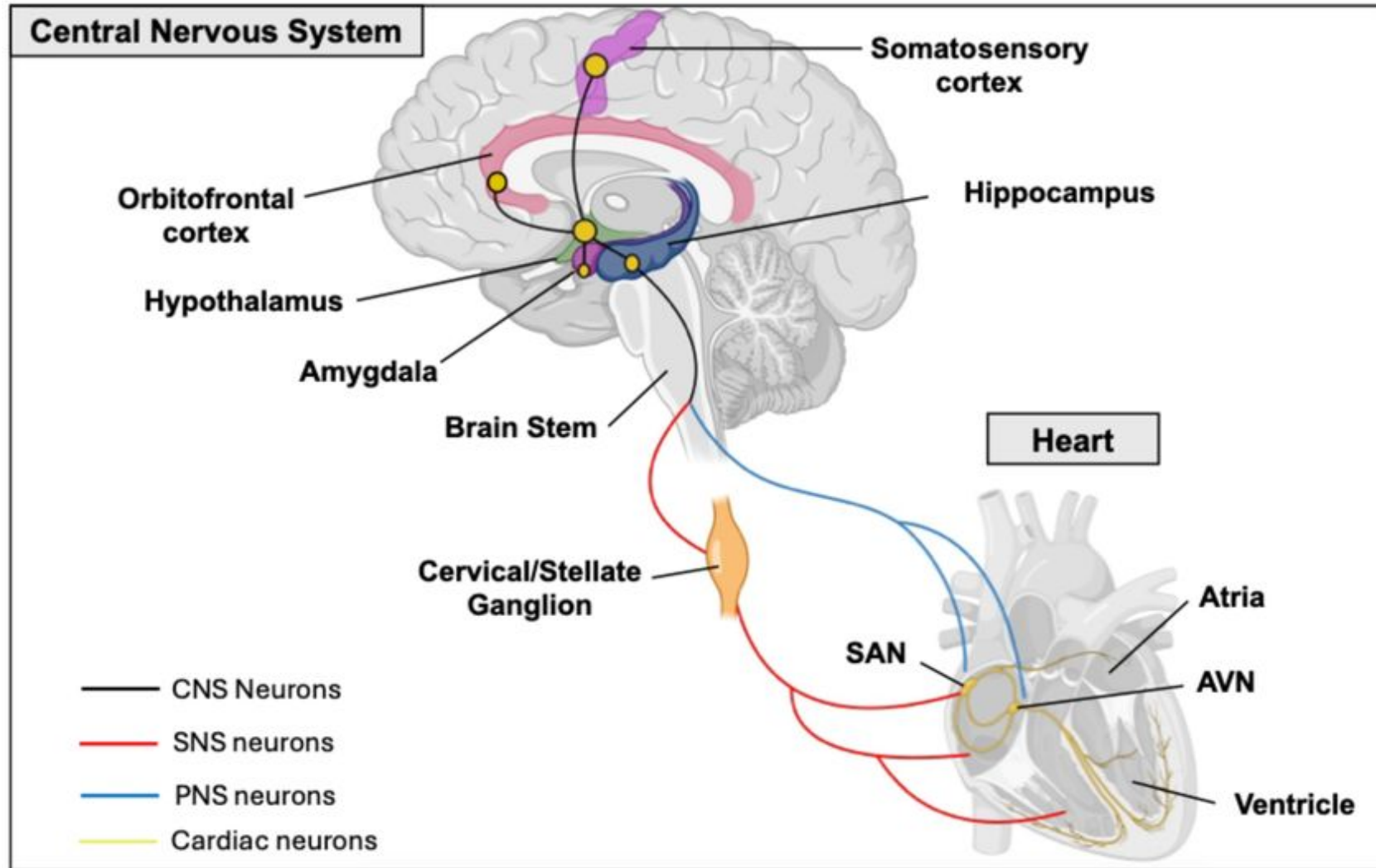
Input



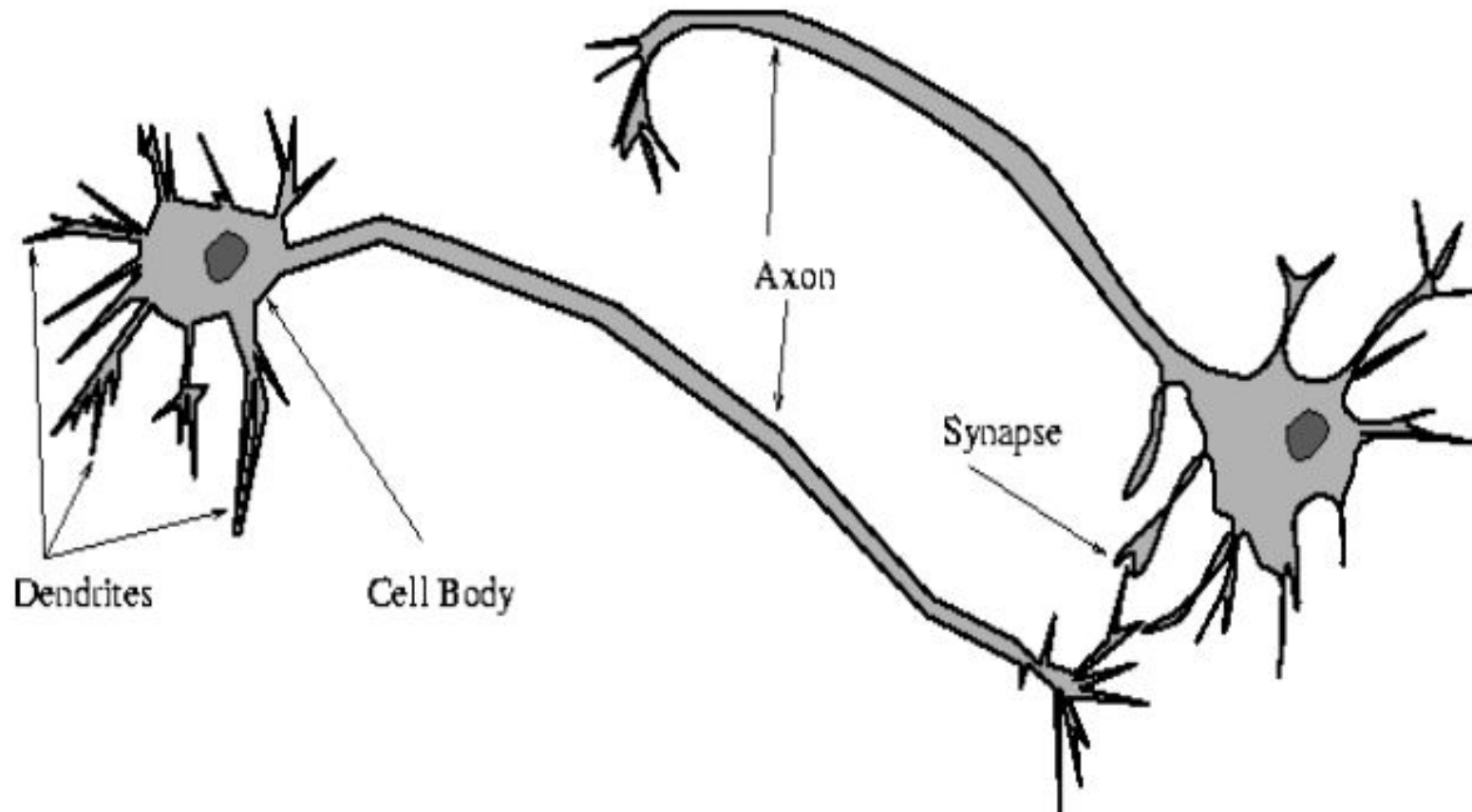
Information flow in nervous system



Information flow in nervous system



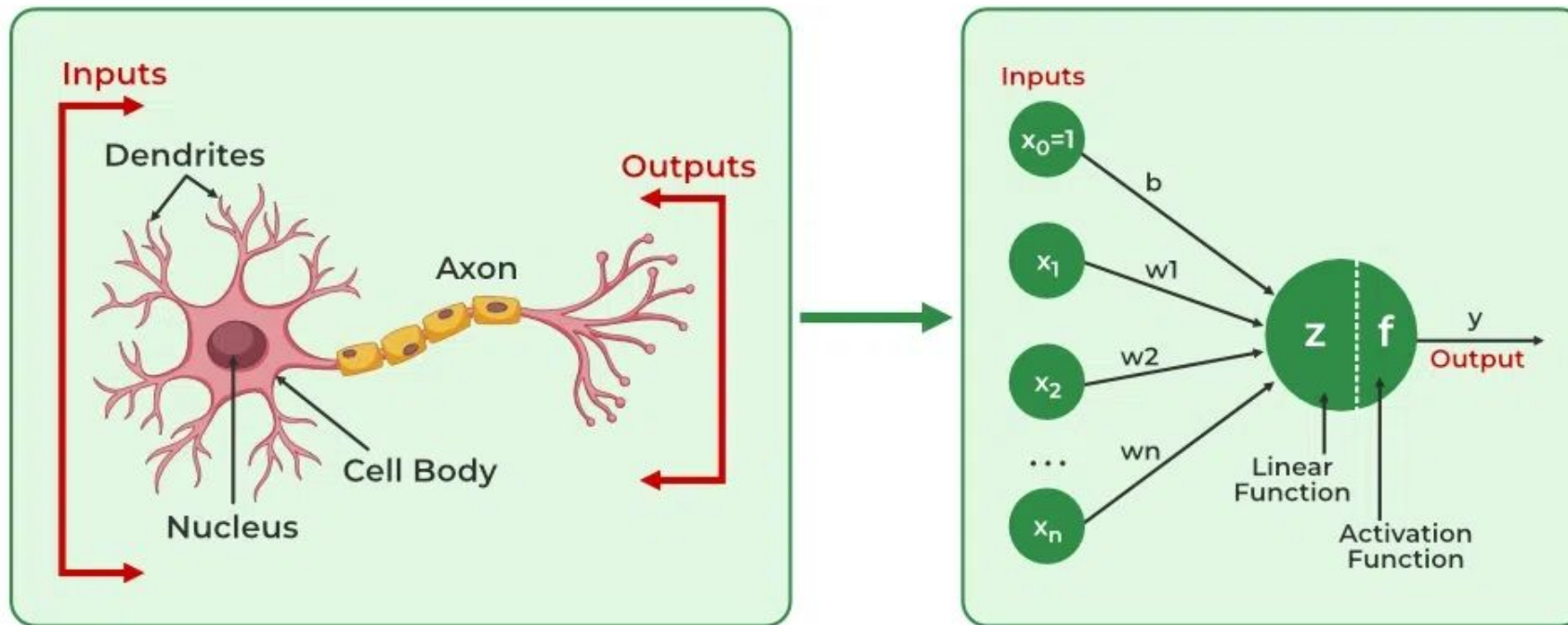
Biological Neural Network



Biological Neural Network

- ▶ Has 3 parts
 - ▶ Soma or cell body:- cell nucleus is located
 - ▶ Dendrites:- nerve connected to cell body
 - ▶ Axon: carries impulses of the neuron
- ▶ End of axon splits into fine strands
- ▶ Each strand terminates into a bulb-like organ called synapse
- ▶ Electric impulses are passed between the synapse and dendrites
- ▶ Synapses are of two types
 - ▶ Inhibitory:- impulses hinder the firing of the receiving cell
 - ▶ Excitatory:- impulses cause the firing of the receiving cell
- ▶ Neuron fires when the total of the weights to receive impulses exceeds the threshold value during the latent summation period
- ▶ After carrying a pulse an axon fiber is in a state of complete non-excitability for a certain time called the refractory period.

Artificial neurons vs Biological neurons



Artificial neurons vs Biological neurons

Aspect	Biological Neurons	Artificial Neurons
Structure	Dendrites: Receive signals from other neurons.	Input Nodes: Receive data and pass it on to the next layer.
	Cell Body (Soma): Processes the signals.	Hidden Layer Nodes: Process and transform the data.
	Axon: Transmits processed signals to other neurons.	Output Nodes: Produce the final result after processing.
Connections	Synapses: Links between neurons that transmit signals.	Weights: Connections between neurons that control the influence of one neuron on another.
Learning Mechanism	Synaptic Plasticity: Changes in synaptic strength based on activity over time.	Backpropagation: Adjusts the weights based on errors in predictions to improve future performance.
Activation	Activation: Neurons fire when signals are strong enough to reach a threshold.	Activation Function: Maps input to output, deciding if the neuron should fire based on the processed data.

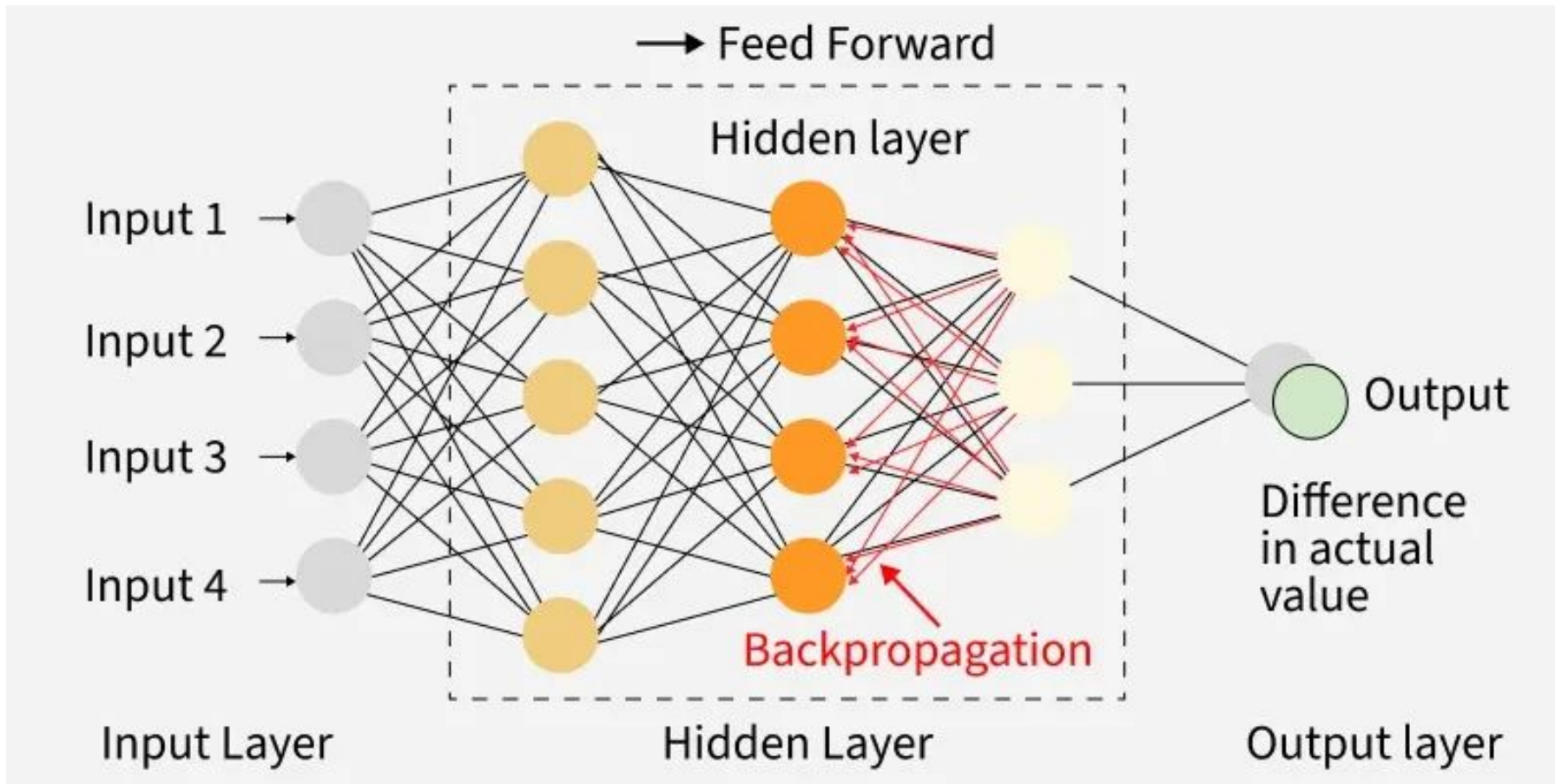
Working of Artificial Neural Networks

- ▶ ANNs work by learning patterns in data through a process called training. During training, the network adjusts itself to improve its accuracy by comparing its predictions with the actual results.
- ▶ **Input Layer:** Data such as an image, text or number is fed into the network through the input layer.
- ▶ **Hidden Layers:** Each neuron in the hidden layers performs some calculation on the input, passing the result to the next layer. The data is transformed and abstracted at each layer.
- ▶ **Output Layer:** After passing through all the layers, the network gives its final prediction like classifying an image as a cat or a dog.
- ▶ **Training and Testing:**
 - ▶ During training, the network is shown examples like images of cats and learns to recognize patterns in them.
 - ▶ After training, the network is tested on new data to check its performance. The better the network is trained, the more accurately it will predict new data.
- ▶ **Backpropagation:** The process used to adjust the weights between neurons. When the network makes a mistake, the weights are updated to reduce the error and improve the next prediction.

How do Artificial Neural Networks learn?

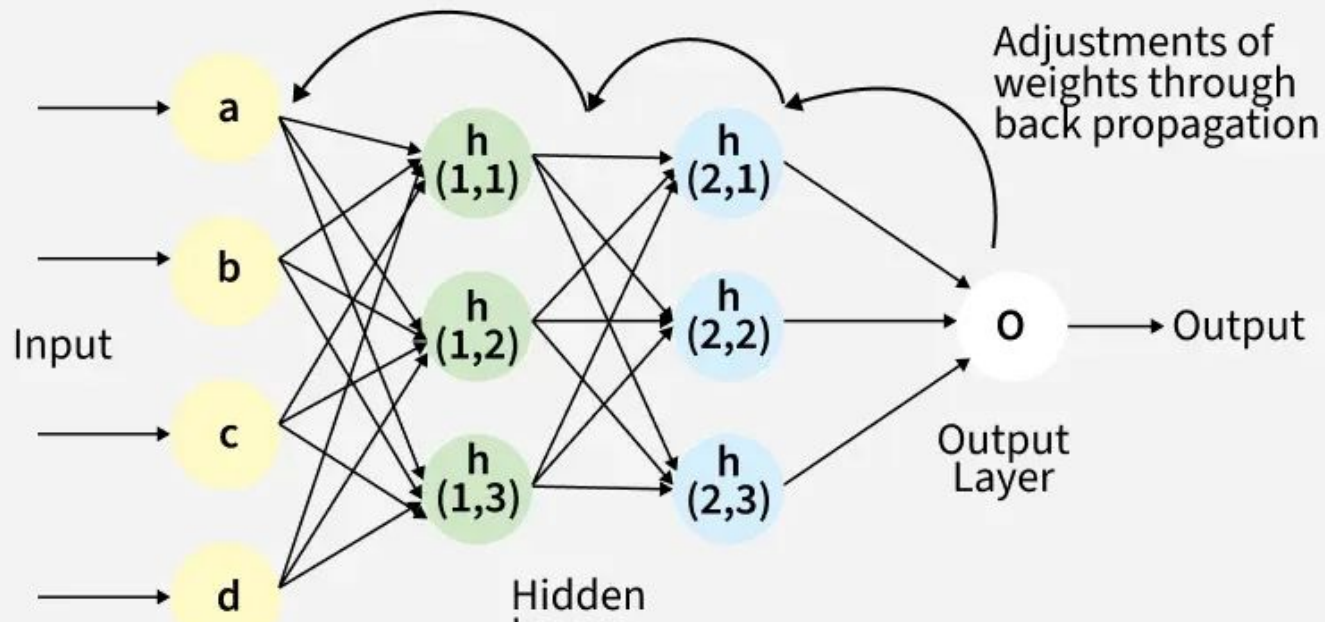
- ▶ Artificial Neural Networks (ANNs) learn by training on a set of data. For example, to teach an ANN to recognize a cat, we show it thousands of images of cats. The network processes these images and learns to identify the features that define a cat.
- ▶ Once the network has been trained, we test it by providing new images to see if it can correctly identify cats. The network's prediction is then compared to the actual label (whether it's a cat or not). If it makes an incorrect prediction, the network adjusts by fine-tuning the weights of the connections between neurons using a process called backpropagation. This involves correcting the weights based on the difference between the predicted and actual result.
- ▶ This process repeats until the network can accurately recognize a cat in an image with minimal error. Essentially, through constant training and feedback, the network becomes better at identifying patterns and making predictions.

How do Artificial Neural Networks learn?

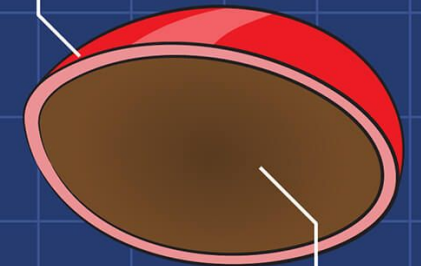


Backpropagation in Neural Network

- ▶ Backpropagation, short for Backward Propagation of Errors, is a key algorithm used to train neural networks by minimizing the difference between predicted and actual outputs. It works by propagating errors backward through the network, using the chain rule of calculus to compute gradients and then iteratively updating the weights and biases. Combined with optimization techniques like gradient descent, backpropagation enables the model to reduce loss across epochs and effectively learn complex patterns from data.



candy-coating shell
(outside function)



chocolate filling
(inside function)

Backpropagation in Neural Network

- ▶ **Efficient Weight Update:** It computes the gradient of the loss function with respect to each weight using the chain rule making it possible to update weights efficiently.
- ▶ **Scalability:** The Back Propagation algorithm scales well to networks with multiple layers and complex architectures making deep learning feasible.
- ▶ **Automated Learning:** With Back Propagation the learning process becomes automated and the model can adjust itself to optimize its performance.

Backpropagation in Neural Network

- ▶ Propagation algorithm involves two main steps: the Forward Pass and the Backward Pass.
- ▶ **Forward Pass Work**
 - ▶ In forward pass the input data is fed into the input layer. These inputs combined with their respective weights are passed to hidden layers. For example in a network with two hidden layers (h1 and h2) the output from h1 serves as the input to h2. Before applying an activation function, a bias is added to the weighted inputs.
 - ▶ Each hidden layer computes the weighted sum ($\sum$) of the inputs then applies an activation function like ReLU (Rectified Linear Unit) to obtain the output (σ). The output is passed to the next layer where an activation function such as softmax converts the weighted outputs into probabilities for classification.

Backpropagation in Neural Network

➔ Backward Pass

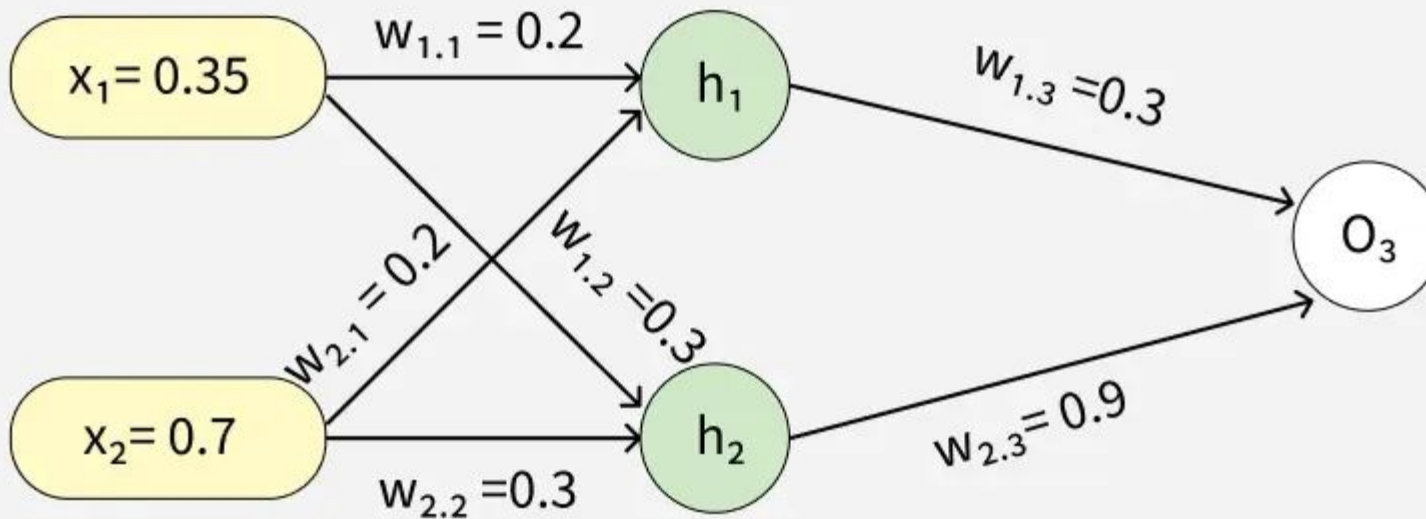
- ▶ In the backward pass the error (the difference between the predicted and actual output) is propagated back through the network to adjust the weights and biases. One common method for error calculation is the Mean Squared Error (MSE) given by:

$$MSE = (\textit{Predicted Output} - \textit{Actual Output})^2$$

- ▶ Once the error is calculated the network adjusts weights using gradients which are computed with the chain rule. These gradients indicate how much each weight and bias should be adjusted to minimize the error in the next iteration. The backward pass continues layer by layer ensuring that the network learns and improves its performance. The activation function through its derivative plays a crucial role in computing these gradients during Back Propagation.

Backpropagation in Neural Network

- Assume the neurons use the sigmoid activation function for the forward and backward pass. The target output is 0.5 and the learning rate is 1.

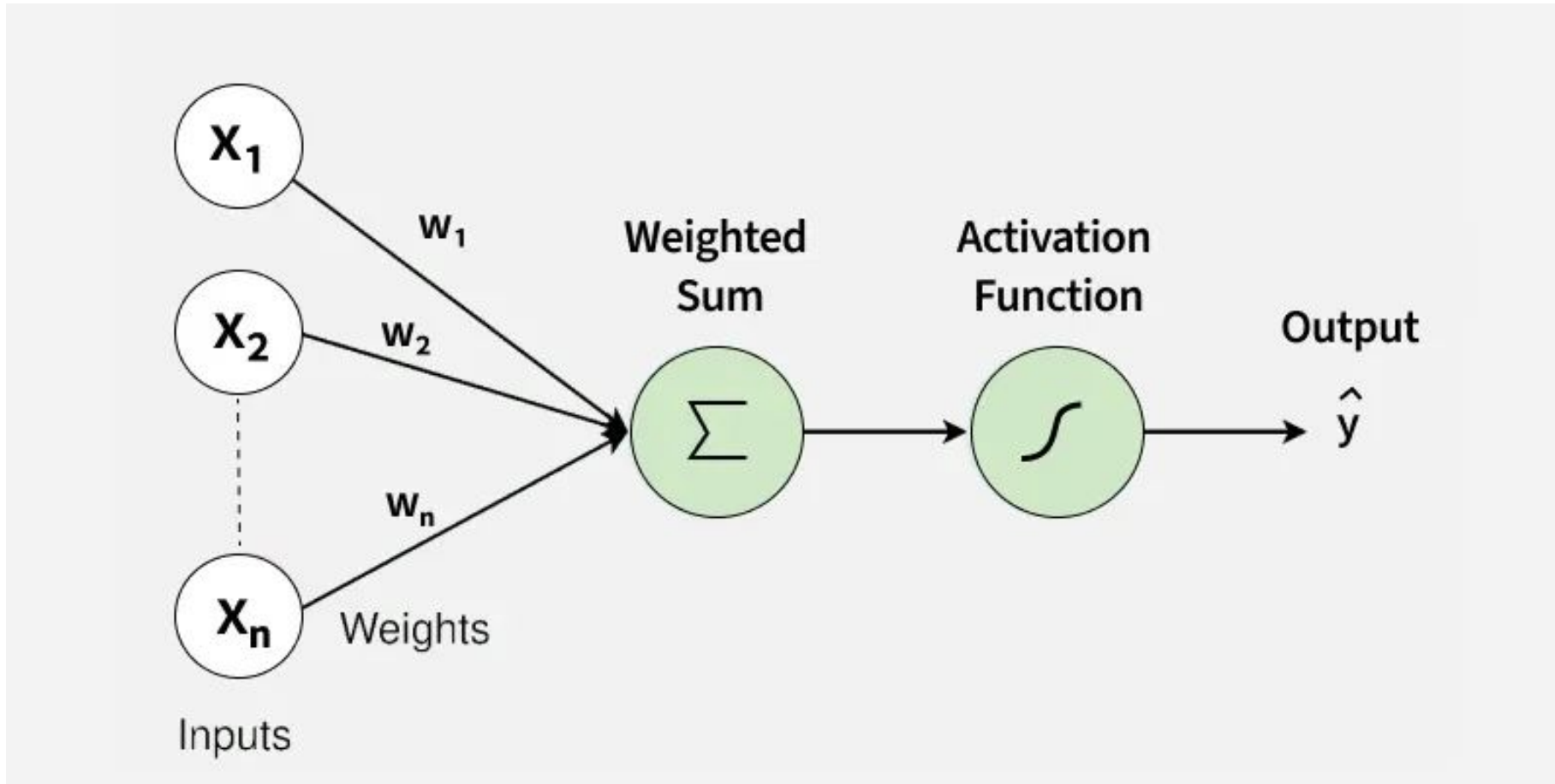


Perceptron model



Camels are the little white lines whereas black lines are shadows in the picture above.

Perceptron model



Perceptron model

- A Perceptron is the simplest form of a neural network that makes decisions by combining inputs with weights and applying an activation function. It is mainly used for binary classification problems. It forms the basic building block of many deep learning models.
 - ▶ Takes multiple inputs and assigns weights
 - ▶ Computes a weighted sum and applies a threshold
 - ▶ Outputs either 0 or 1 (binary outcome)
 - ▶ Forms the foundation of larger neural networks
- ▶ Inputs $(x_1, x_2, \dots, x_n)$ are the features or measurable attributes of a data point that the perceptron uses to make a decision. Each input provides a signal that contributes to the final output.
- ▶ Inputs themselves have no inherent influence unless multiplied by weights.
- ▶ Example: For an OR gate, the inputs are binary: $(x_1, x_2) \in \{0,1\}^2$

Perceptron model

- ▶ Weights ($w_1, w_2, \dots, w_n$) determine how strongly each input contributes to the prediction. A larger weight means the corresponding input has a higher impact.
 - ▶ Weights are learned during training, adjusting based on errors.
 - ▶ They act like importance scores for each feature.
- ▶ Bias (b) is a constant value added to the weighted sum to shift the decision boundary.
 - ▶ It allows the perceptron to classify correctly even when all input features are zero.
 - ▶ Bias controls when the perceptron activates, independent of any input.
 - ▶ Bias ensures the model is not forced to pass the decision boundary through the origin.
- ▶ Net Input (Weighted Sum) is the combined effect of all inputs and their weights:
$$z = \sum_{i=1}^n w_i x_i + b$$
 - ▶ Represents the activation strength before passing through the activation function.
 - ▶ If z is high or low enough, it determines the final class.

Perceptron model

- ▶ Activation Function (Step Function) converts the numerical input into a binary output:

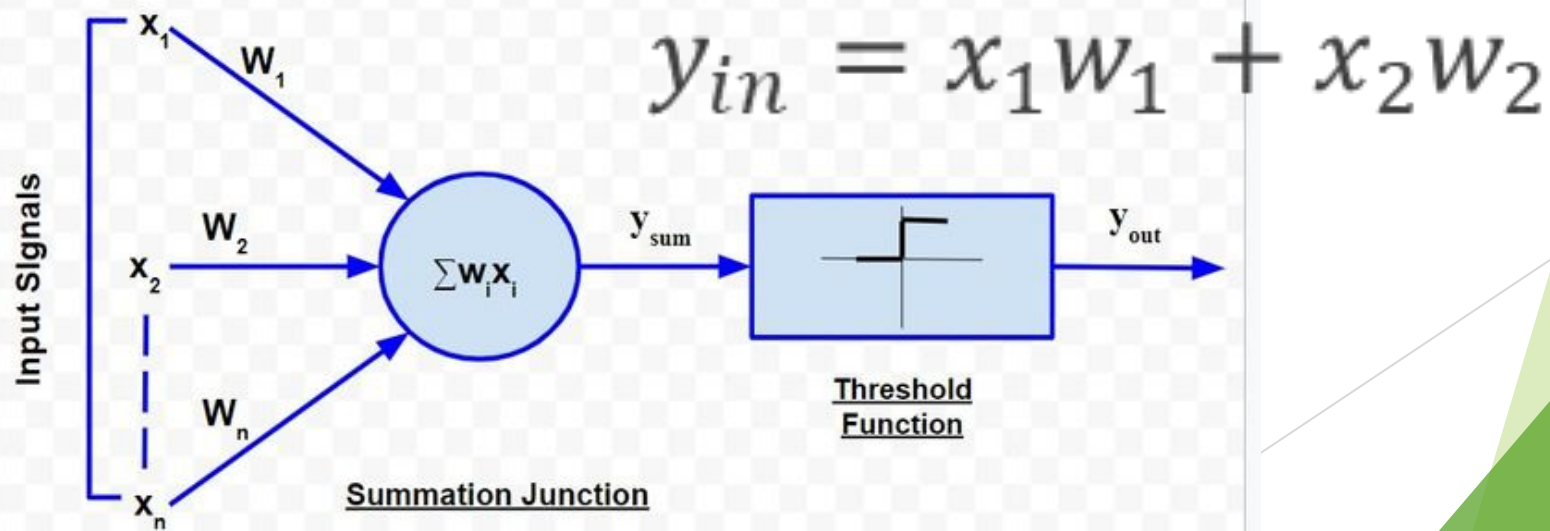
$$\hat{y} = \begin{cases} 1, & \text{if } z \geq 0 \\ 0, & \text{otherwise} \end{cases}$$

- ▶ It introduces non-linearity in the decision-making, although the decision boundary remains linear.
- ▶ Output is always 0 or 1 making perceptrons suitable for binary classification.

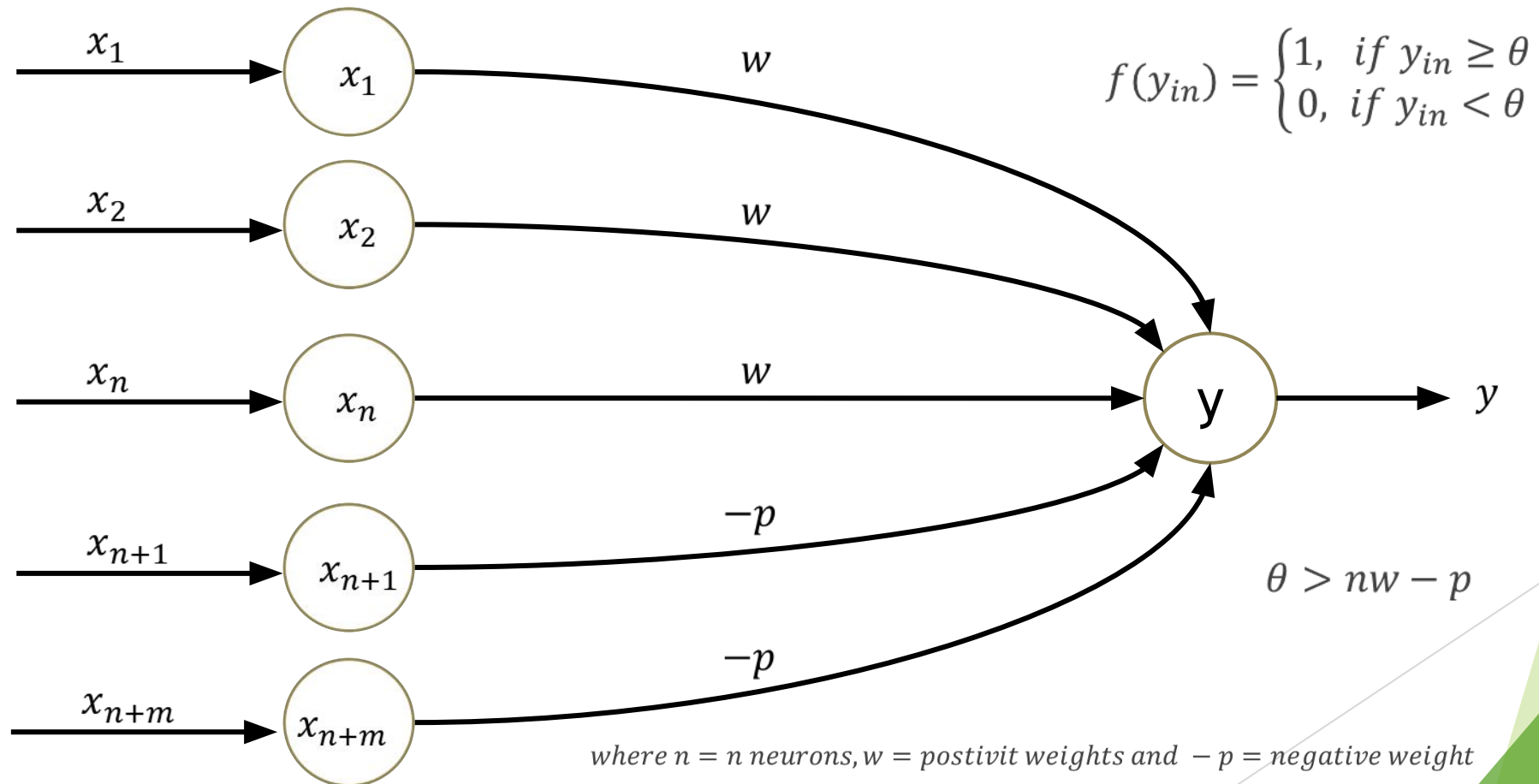


McCulloch-Pitts Model of Neuron

- ▶ Earliest ANN model
- ▶ Has only two types of inputs – **Excitatory (cause)** and **Inhibitory (prevention)**.
- ▶ The excitatory inputs have weights of positive magnitude and the inhibitory weights have weights of negative magnitude.
- ▶ The inputs of the McCulloch-Pitts neuron could be either 0 or 1. It has a threshold function as an activation function.
- ▶ the output signal y_{out} is 1 if the input y_{sum} is greater than or equal to a given threshold value, else 0.



McCulloch-Pitts Model of Neuron



McCulloch-Pitts Model of Neuron

- ▶ Simple McCulloch-Pitts neurons can be used to design logical operations.
- ▶ “John carries an umbrella if it is sunny or if it is raining”. Decide when John will carry the umbrella.
 - ▶ First scenario: It is not raining, nor it is sunny
 - ▶ Second scenario: It is not raining, but it is sunny
 - ▶ Third scenario: It is raining, and it is not sunny
 - ▶ Fourth scenario: It is raining as well as it is sunny
- ▶ X1: Is it raining?
- ▶ X2 : Is it sunny?

$$y_{sum} = \sum_{i=1}^2 w_i x_i$$

$$y_{out} = f(y_{sum}) = \begin{cases} 1, & x \geq 1 \\ 0, & x < 1 \end{cases}$$

McCulloch-Pitts Model of Neuron

Situation	x_1	x_2	y_{sum}	y_{out}
1	0	0	0	0
2	0	1	1	1
3	1	0	1	1
4	1	1	2	1

McCulloch-Pitts Model of Neuron

➤ The OR gate outputs 1 if ANY input is active (at least one input is 1).

➤ **Given:**

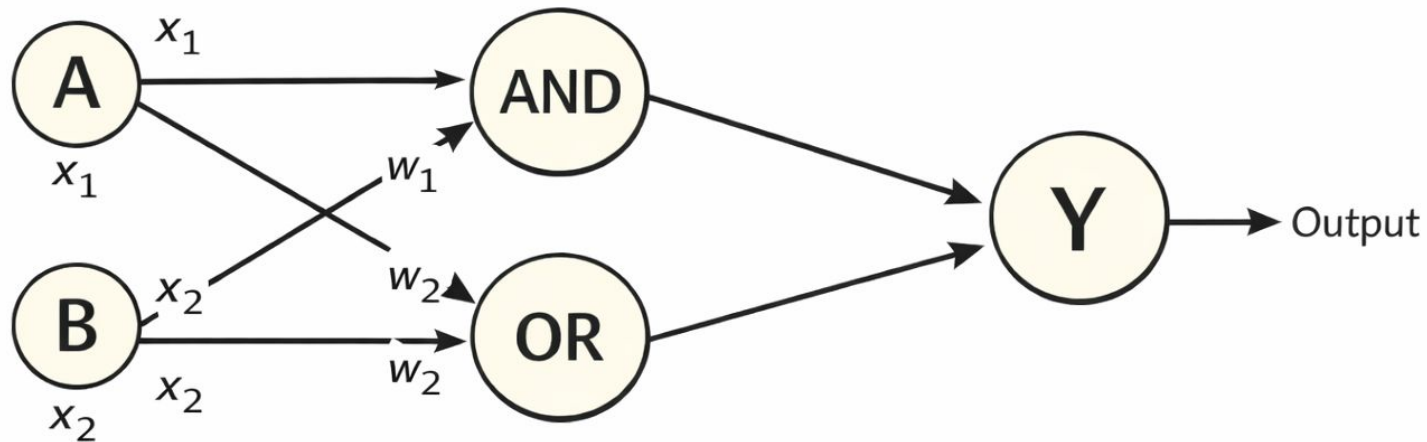
- 2 inputs: x_1, x_2
- Binary inputs: 0 or 1
- Weights: $w_1 = 1, w_2 = 1$ (equal weights)
- What would be considered as ideal threshold?

➤ $y_{in} = x_1w_1 + x_2w_2 = x_1(1) + x_2(1) = x_1 + x_2$

x_1	x_2	$y_{in} = x_1 + x_2$		Output (y)
0	0	0	No	0
0	1	1	Yes	1
1	0	1	Yes	1
1	1	2	Yes	1

McCulloch-Pitts Model of Neuron

- ▶ $Y = (A \text{ AND } B) \text{ AND } (A \text{ OR } B)$, $w_1, w_2=1$
- ▶ $\theta_{\text{AND}}=2$, $\theta_{\text{OR}}=1$, $\theta_Y=2$
- ▶ $z_1=\text{AND}$, $z_2=\text{OR}$, $\text{net}Y=z_1+z_2$



McCulloch-Pitts Model of Neuron

- ▶ A wind turbine has a safety system which uses three inputs to a logic circuit. A certain combination of conditions results in an output, X , from the logic circuit being equal to 1. When the value of $X = 1$ then the wind turbine is shut down.
- ▶ The following table shows which parameters are being monitored and form the three inputs to the logic circuit.

parameter description	parameter	binary value	description of condition
turbine speed	S	0	turbine speed ≤ 1000 rpm
		1	turbine speed > 1000 rpm
bearing temperature	T	0	bearing temperature $\leq 80^{\circ}\text{C}$
		1	bearing temperature $> 80^{\circ}\text{C}$
wind velocity	W	0	wind velocity ≤ 120 kph
		1	wind velocity > 120 kph

McCulloch-Pitts Model of Neuron

- ▶ The output, y , will have a value of 1 if any of the following combination of conditions occur:
 - ▶ **Either** turbine speed ≤ 1000 rpm and bearing temperature $> 80^{\circ}\text{C}$
 - ▶ **Or** turbine speed > 1000 rpm and wind velocity > 120 kph
 - ▶ **Or** bearing temperature $\leq 80^{\circ}\text{C}$ and wind velocity > 120 kph

$$Y = (S \text{ AND } T) \text{ OR } (S \text{ AND } W) \text{ OR } (T \text{ AND } W)$$

McCulloch-Pitts Model of Neuron

➤ XOR function

➤ $y = x_1 \bar{x}_2 w_1 + \bar{x}_1 x_2 w_2$

➤ $y = z_1 + z_2$

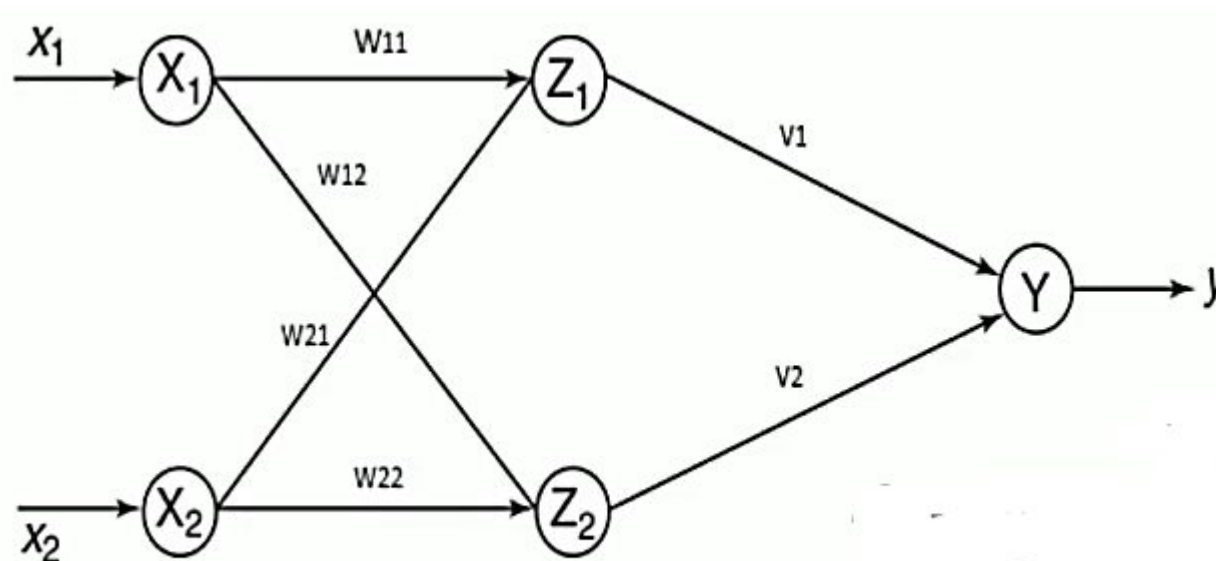
➤ Where

➤ $z_1 = x_1 \bar{x}_2$

➤ $z_2 = \bar{x}_1 x_2$

➤ $y = z_1 \text{ OR } z_2$

0	0	0
0	1	1
1	0	1
1	1	0



McCulloch-Pitts Model of Neuron

► $z_1 = x_1 \overline{x_2}$

► *Let* $w_1 = w_2 = 1$

► $(0,0), z_{in} = 0 * 1 + 0 * 1 = 0$

► $(0,1), z_{in} = 0 * 1 + 1 * 1 = 1$

► $(1,0), z_{in} = 1 * 0 + 1 * 1 = 1$

► $(1,1), z_{in} = 1 * 1 + 1 * 1 = 2$

► Hence, $w_1=w_2=1$ will not work here.

► Check $w_1=1, w_2=-1$

0	0	0
0	1	0
1	0	1
1	1	0